

Graduate Operating Systems

1/5
ECS 251
Amanda Raybuck

Course format, slides, and materials adapted from courses taught by Sam King, Amy Ousterhout, Tianyin Xu, and others

Agenda for Today

- Brief Introductions
- Course logistics
- Introduction to OS and systems research
- How to read a paper

About Me

- Amanda Raybuck, but you can call me Amanda
- Got my BSc in CS from UNLV in May 2017 
- Got my PhD in CS from UT Austin in May 2024 
- Second-year Assistant Professor at UC Davis 
- Research focus: Systems!
 - Tiered/heterogeneous memory management, microsecond-scale datacenter software

Meet your TA

Arnav Tope

Course Logistics

- Graduate-level, research-focused course
- The goals of the course are:
 - To learn about classic and cutting-edge OS techniques
 - To practice reading and discussing research/technical papers
 - To conduct a research project
- Three main components:
 - Reading list + in-class quizzes + discussion
 - Leading a lecture
 - Research project
- Discussion session:
 - First half of quarter: Practical skills and aspects of systems research
 - Second half of quarter: Project group meetings

Approach to the Class

- Discussion based rather than lecture based
 - Lectures are boring and it's prime afternoon-nap time :)
 - Help more deeply understand the topics
- 2 phase discussion format
 - Smaller group discussions first, crystalize ideas and thoughts on the paper
 - Then, full class discussion
 - May be a bit chaotic, but let's try it out!

OS Topics

- 5 main topics in this course:
 - Core OS design: Historical context and modern designs
 - Scheduling and process management
 - Memory management
 - File systems and data management
 - Concurrency
- General intro & 3-4 papers on each topic
- Topics not covered (but still important and cool systems topics):
 - Virtualization
 - Distributed Systems
 - Security
 - Verification/correctness
 - MLSys or SysML

Readings, Discussion, and Quizzes

- We will read 1-2 papers per week
- Everyone should read the papers before class
 - Come prepared to discuss!
- Class format:
 - Introduction to the problem and background
 - Core idea(s) of the research paper and its implementation
 - Small group discussions
 - Class-wide discussion
- 8 short quizzes, roughly 1/week, on previous readings
 - Lowest quiz score will be replaced at end of quarter with 100%

Leading a Lecture

- Each student will help lead a discussion of a research paper
 - Done in groups of 3-4 students
- Pick a paper by end of day Friday (1/9)
- Preparation:
 - Read paper as usual
 - Prepare slides presenting the paper
 - Prepare some thought-provoking, open-ended discussion topics related to the paper
 - Prepare 4 potential quiz questions for the paper
 - Send slides and quiz questions to me by **noon the day before** your lecture

Paper Presentations

- Feel free to borrow slides, but make sure to credit!
- Suggested outline for your presentations:
 - 5 minutes on background
 - 10 minutes on the “meat” of the paper
 - 10 minutes for smaller, group-based discussions
 - 15 minutes for class-wide discussion
- Your grade will be based on:
 - Content and polish of your presentation
 - Discussion questions you have for the class (needs to be thoughtful, thought provoking, and related to the paper)

Research Project

- Open-ended research project
 - Done in groups of 3-4 students
 - You choose the topic! (Related in some way to systems, broadly construed)
 - Unknown answers, advances the state-of-the-art (probably in a small way)
- Goals:
 - Get experience with the research cycle (propose, conduct, write, present)
 - Gain experience with writing real systems code

Broad Project Categories

- **Study:** Qualitatively or quantitatively analyze an important aspect of a type of system
- **Measurement:** Measure/characterize an important aspect of a type of system through experiments
- **Tooling:** Design and implement a new tool that can address an important problem in modern systems
- **System:** Design and implement a novel system with new capabilities or properties

We will brainstorm potential project ideas in the next couple of weeks

Project Schedule

- **1/21:** Group membership due
- **1/26:** Project choice due
- **2/2:** Project proposal due
- **2/9 - 3/2:** Project group meetings during discussion sessions
- **3/9 - 3/13:** Project presentations
- **3/18:** Final project submission

More on each component to come later in the quarter

Grading

Straight grading scale ($\geq 93\%$ A, 90-92 A-, 87-89 B+, 83-86 B, 80-82 B-, etc.)

Item	Percent	Description
Quizzes	32%	Eight quizzes worth 4% each
Research Project	48%	Conduct an open-ended research project related to systems
Lead a Lecture	20%	Lead a lecture on a research paper

Academic Integrity and Generative AI Policy

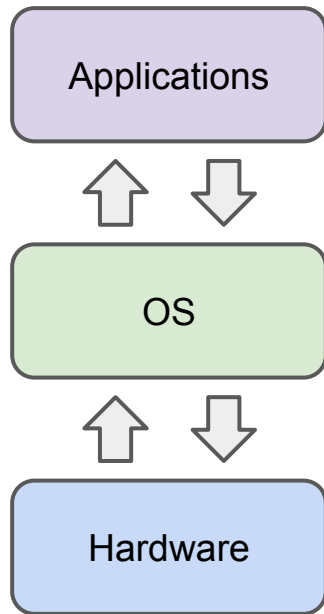
- Talking with others in the class about papers, projects is OK, encouraged
- Give credit to any resources you borrow for presentations, write-ups
 - E.g., if you use slides from a conference presentation in your presentation, give credit
 - When in doubt, cite it!
- I am willing to allow the use of Generative AI, but within reason (& ACK use)
 - Using it for proposal/project writing:
 - You may use it for a “first draft” but I expect human-made revisions over what the model produces
 - OK to use for grammar and clarity improvements
 - Using it for code generation:
 - OK , but acknowledge use in write-ups
 - You (and your group) are responsible for understanding anything you turn in!

Why Study (Operating) Systems?

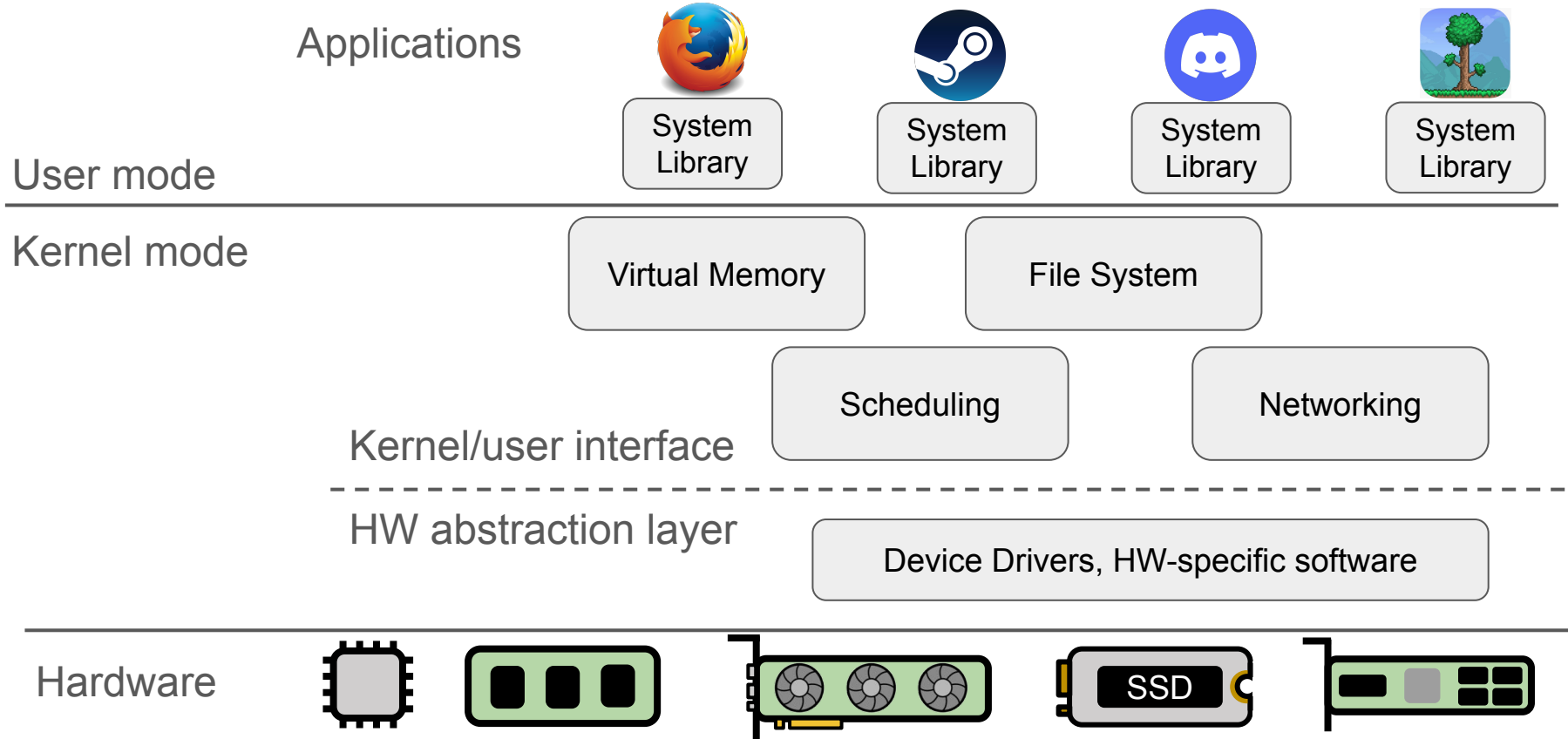
- Gain a full stack understanding
 - Understand abstractions and principles
 - Understand how the resources your program needs are managed
 - Why does my program have the performance it does? How can I make it better?
 - Why does my program crash?
- Improve coding skills
 - Especially lower level skills (e.g., understanding memory & pointers)
 - Not necessarily kernel hacking, systems research is much broader than that!
- Related to a lot of other areas in CS:
 - Architecture, Programming Languages, Software Engineering, Networking, Embedded/Mobile Computing, HPC, ...

Introduction to Operating Systems

- What is an operating system?
 - Systems software that:
 - Manages computer hardware and software
 - Provides services to applications
 - Acts as an interface between applications and hardware
- Highly practical side of CS
 - Emphasis on practicality and novelty
 - Research ideas are actually implemented and evaluated
- Just another computer program that someone (maybe you!) wrote



Operating Systems: More Depth



Operating Systems: More Depth

- Allows user programs to deal with simpler abstractions
 - Virtual addresses rather than physical addresses
 - Files rather than disk blocks
- Hides complexity of hardware
 - Can hide failures, ensure reliability
 - Hides heterogeneity of hardware (e.g., file abstraction is the same regardless of SSD or HDD)
- Provides the illusion that an application is the only one running
 - Has access to “infinite” memory
 - Isolated and protected from other applications on the system
- We'll learn more about some of these in Wednesday's paper (UNIX)

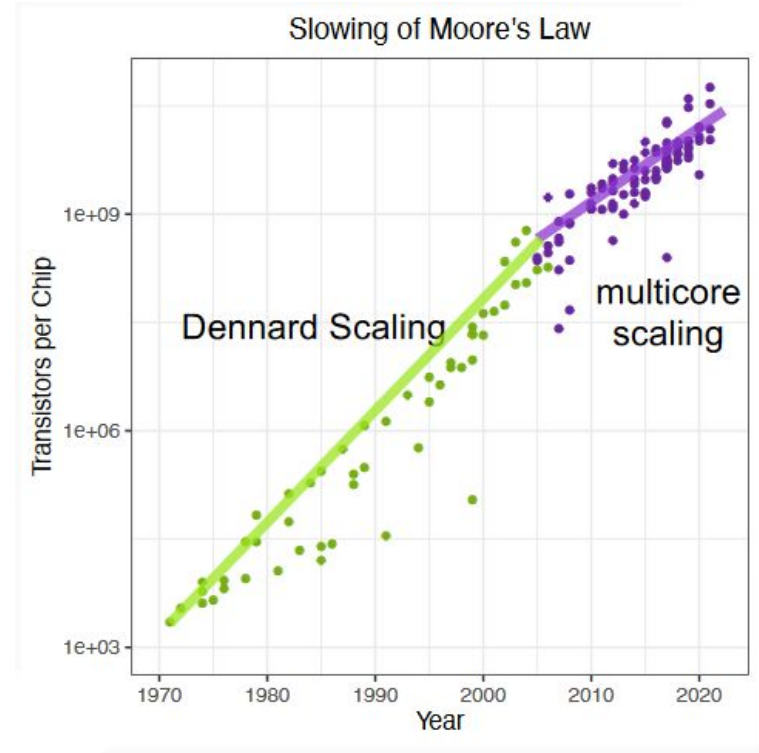
Why Do Research in (Operating) Systems?

- Lots of interesting things these days...
 - Long-standing trends have ended or are slowing (Dennard Scaling, Moore's Law)
 - Old assumptions are being challenged
- New, better, faster hardware
 - Multicore CPUs, larger memories, fast storage and networking, beefy GPUs, ...
- New, bigger, more complex applications
 - Large distributed systems, machine learning, complex databases
 - Demand high performance from the hardware and system
- Increasing scale (data centers)
 - Many thousands of machines, perhaps geographically distributed

How should systems adapt?

Trend: The Slowing of Moore's Law

- Ever increasing demand for compute
- Faster CPUs every few years!
 - Moore's Law: Transistors double every 2 years
 - Dennard Scaling: Transistors get smaller but power density remains the same
- But, this is slowing/ending
- Consequences:
 - More cores on a die (multicore)
 - Move tasks to more specialized hardware/custom accelerators



Trend: Better Hardware

A typical high-end machine comparison:

	1980	Today	Ratio
Transistors	100K	2B	~20,000X
Cores X Clock Speed	1 X 10 Mhz	8 X 2.5-5 GhZ	~3,000X
DRAM	1 MB	64 GB	64,000X
Storage	5 MB	1 TB	200,000X
Network BW	10Mbits/sec	10GBits/sec	1000X
\$/Machine	\$30K	1.5K	1/20X

Trend: Increasingly Complex Applications

- Larger, more complex applications that make use of lots of resources
- Typically user-facing applications with strict performance requirements



Gemini



redis



mongoDB



Why do Systems Research if I'm Not a Systems Person?

Doing systems research can help you gain a lot of useful skills:

- Measuring and understanding a problem before you begin
 - Design and perform empirical studies to understand a problem and a solution
- Combining many existing techniques and solutions to solve a new problem
 - You may see a lot of techniques in systems used over and over again
 - Gain creativity with combining various techniques
- Getting comfortable with tradeoffs and uncertainty
 - “It depends...” is a common answer in systems (with elaboration)
 - Different assumptions and different points in the tradeoff space will lead to different solutions
 - Gain skills in balancing these tradeoffs (e.g., raw performance vs. programmability and ease of use of a system)

How to Read a Paper

- Takes time to develop this skill
 - Reading papers can take a long time at first, but it gets faster
 - Takes practice!
- Take notes if needed while reading
 - Often helpful to print out the paper, underline/highlight sentences, write in the margins
 - Handwritten notes on the paper itself or on a separate piece of paper vs. typed notes
 - Mainly a preference thing
 - I find handwritten notes help me have a better understanding
 - But typed notes can be reorganized and restructured as needed
- Systems research papers often have a defined structure:
 - Abstract, Intro, Background, Design, Implementation, Evaluation, Related Work, Conclusion

Systems Paper Structure: Abstract and Introduction

- Abstract and introduction summarize the paper
- If you understand the introduction, you'll understand:
 - The context of the work
 - The intellectual contributions
 - The value of those contributions
- Reading the introduction allows you to:
 - Understand the problem the paper is solving
 - Understand the basic idea of the solution, but not the technical details

Systems Paper Structure: Related, Prior Work

- Background and Related Works section
 - Background usually appears at the beginning of the paper, sets context
 - Related work can be in beginning or end of the paper
- Overview of the works the paper build on and its place in that context
- **Related Work:** Other pieces of research closely related to this paper
- **Prior Work:** Work this paper directly builds on

Systems Paper Structure: Technical Sections & Evaluation

- Technical sections (Design and Implementation):
 - Describes how the system or technique works and is built
 - Contains more details about how it solves the problem outlined in the Intro
- Evaluation:
 - Quantitative results from a prototype implementation and/or simulation
 - Usually a combination of microbenchmarks and macrobenchmarks
 - Microbenchmarks: Simpler experiments designed to showcase a specific aspect of the system (e.g., performance of page fault handling)
 - Macrobenchmarks: More realistic workloads or applications designed to show the real benefits of the system (e.g., overall speedup offered to the Redis application)

The Three-Pass Approach

- Don't necessarily start at page 1 and trudge through till the end
 - Instead, read the paper in multiple passes, focusing on a different goal on each pass
- First pass: Get a general idea about the paper
- Second pass: Grasp the paper's content but not its details
- Third pass: Deeply understand the details

The Three-Pass Approach: First Pass

- Time: Roughly 10 minutes
- Get a rough sense of the paper
 - Read the title, abstract, and introduction
 - Read the section and subsection headings (headings only)
 - Read the conclusion
 - Scan the references
- Goal: Answer the Five Cs”
 - Category: What kind of paper is this (measurement, system analysis, prototype)?
 - Context: What are the related papers? What is the theoretical/practical basis for the problem?
 - Correctness: Does the paper appear to be valid?
 - Contributions: What are the main contributions?
 - Clarity: Is the paper well written?

The Three-Pass Approach: Second Pass

- Time: Roughly an hour
- Read the paper with greater care
 - Read the text of the paper, but ignore the finer details
 - Look carefully at figures and graphs. What do they show?
 - Make a note of any related work you may want to check out
- Goal: Understand the paper well enough to summarize it to someone else
 - Including the main idea, insights, and supporting evidence

The Three-Pass Approach: Third Pass

- Time: >1 hour
- Read carefully with an eye for *virtually reimplementing* the paper
 - Understand the assumptions of the authors, how would you recreate the work?
 - Find the innovations of the paper, but also shortcomings and unstated assumptions
 - Jot down any ideas for future work
- Goal: Deep understanding of the paper
 - Identify its strong and weak points

An Alternative (or Complementary) Approach:

- Ask yourself questions when reading the paper
- High level questions:
 - What is the problem? Why is it important?
 - What is the solution? What is new about the solution?
 - Which parts did you not understand?

More Detailed Questions

- **Solution:**
 - What is their approach?
 - What are the key components and how important is each one?
 - Did this paper solve the problem?
 - Are there limitations? How fundamental are they?
- **Evaluation:**
 - How did they evaluate their work?
 - Are the experiments realistic?
 - Did they demonstrate that the solution works?
- **Impact:**
 - Do you think this work is impactful? Why (or why not)?
 - What kind of impact do you think it will have?

More Detailed Questions

- Authors:
 - Who are the authors and why did they write this paper now?
- Extensions:
 - What weaknesses does the paper have? How could it be improved?
 - Could you apply these ideas to other problems in other domains?
 - Useful to think of these as a researcher!

TODOs

- Read the UNIX paper
- Start thinking about what paper you want to present